

Поиск наибольшего ациклического подграфа

Золотарева Екатерина

Апрель 2026

Аннотация

Проект посвящен задаче поиска максимального ациклического подграфа в ориентированном графе. Будет показано, что это задача является **NP**-трудной. Далее рассматривается полиномиальный алгоритм, позволяющий найти ациклический подграф, содержащий не менее половины рёбер исходного графа. В частности, такой подграф имеет размер не менее половины от размера максимального ациклического подграфа. Существуют примеры, когда вышеупомянутый алгоритм находит подграф в точности в 2 раза меньшего размера, чем оптимальный ответ.

В проекте будет проведён обзор имеющихся полиномиальных алгоритмов, позволяющих приближать оптимальный ответ с определённой точностью, а также сформулированной гипотезы о неулучшаемости имеющейся оценки в $1/2$.

1 Введение

Задача поиска наибольшего ациклического подграфа (maximum acyclic subgraph) формулируется следующим образом: дан ориентированный граф $G = (V, E)$, не содержащий петель и кратных рёбер. Требуется найти максимальное по размеру $E' \subset E$, такое что подграф $G' = (V, E')$ не содержит ориентированных циклов.

Эта задача напрямую связана с задачей поиска минимального разрезающего набора дуг (Feedback Arc Set). Feedback Arc Set — это задача поиска минимального по размеру множества рёбер в ориентированном графе, такого что любой ориентированный цикл в этом графе содержит хотя бы одно ребро из этого множества. Тогда дополнение к такому множеству рёбер будет образовывать ациклический подграф (действительно, если в дополнении есть цикл, то этот цикл был и в исходном графе и он не пересекается с нашим множеством рёбер - противоречие). Наоборот, дополнение к ациклическому подграфу (то есть E' в исходной формулировке maximum acyclic subgraph) обязано пересекаться с любым циклом в исходном графе G , ведь иначе нашёлся бы цикл, полностью состоящий из рёбер из $E \setminus E'$, то есть нашёлся бы цикл в нашем ациклическом подграфе - противоречие.

NP-трудность задачи Feedback Arc Set (FAS) была показана Ричардом Карпом в статье "Reducibility among Combinatorial Problems"³ в 1972-м году. Если быть точнее, в этой статье была доказана **NP**-полнота соответствующей задачи распознавания (по ориентированному графу и числу k понять, есть ли разрезающий набор дуг размера $\leq k$).

В этом проекте будет предложен простой полиномиальный алгоритм, позволяющий получить ациклический подграф заданного графа, имеющий размер не меньше половины от размера наибольшего ациклического подграфа. Однако, несмотря на простоту этого алгоритма, существенных улучшений этой оценки добиться так и не удалось. В 1990-м году Бонни Бергер и Питер Шор¹ предъявили полиномиальный алгоритм (сначала недетерминированный, затем, в той же статье, и детерминированную версию), позволяющий найти ациклический подграф, содержащий хотя бы $(1/2 + \Omega(1/\sqrt{\Delta(G)})) \cdot |E|$, где $\Delta(G)$ - это максимальная из степеней вершин G .

В 2004 году Венкат Гурусвами, Прасад Рагхавендра и Риши Сакет² выполнили полиномиальное сведение, сохраняющее аппроксимацию, и свели аппроксимацию Unique Label Cover к аппроксимации Maximum Acyclic Subgraph. Задача Unique Label Cover: дан неориентированный граф, каждому ребру (u, v) которого сопоставляется перестановка π_{uv} множества меток $\{1, \dots, k\}$. Нужно каждой вершине v присвоить метку $L(v)$ так, чтобы удовлетворить как можно больше условий вида $L(v) = \pi_{uv}(L(u))$. В 2002 году Субхашем Хотом была сформулирована "Гипотеза об уникальных играх", которая утверждает, что $\forall \epsilon, \delta > 0$ при достаточно большом k , если нам поступает на вход граф, и гарантируется, что реализуется один из случаев:

1. Существует расстановка меток, удовлетворяющая хотя бы $1 - \epsilon$ доле условий
2. Не существует расстановки меток, удовлетворяющей хотя бы δ доле условий, то задача распознавания, какой именно из случаев (1) или (2) реализуется, является **NP**-трудной.

Венкат Гурусвами, Прасад Рагхавендра и Риши Сакет² доказали, что $\forall \epsilon > 0$ если верна Гипотеза об уникальных играх, то не существует полиномиального алгоритма, который принимает граф, в котором гарантированно есть ациклический подграф, содержащий хотя бы $(1 - \epsilon)$ -долю рёбер исходного графа, и выдаёт ациклический подграф, содержащий хотя бы $1/2 + \epsilon$ рёбер. Отсюда следует, что если верна Гипотеза об уникальных играх, то $\forall \epsilon > 0$ задача аппроксимации MAS с точностью $1/2 + \epsilon$ является **NP**-трудной.

2 NP-трудность Maximum Acyclic Subgraph

Тут может возникнуть небольшая путаница в терминах, ведь класс **NP** состоит из задач распознавания, и классическое определение **NP**-трудности тоже подразумевает, что соответствующая задача - задача распознавания.

Поэтому мы будем использовать следующее определение: Задача A называется **NP**-трудной, если для любой задачи $B \in \mathbf{NP}$: B полиномиально сводится к A . Где под полиномиальной сводимостью подразумевается, что

B решается за полиномиальное время на Маине Тьюринга с доступом к аракуду A .

Понятно, что если мы умеем решать задачу поиска максимального ациклического подграфа, то мы умеем решать соответствующую задачу распознавания: по ориентированному графу $G = (V, E)$ и числу k проверить, если ли в G ациклический подграф, состоящий из k рёбер.

Поэтому для того, чтобы показать **NP**-трудность задачи поиска максимального ациклического подграфа, сведём к соответствующей задаче распознавания **NP**-полную задачу $\text{INDSET} = \{(G_1, k_1) : \text{проверить, правда ли } \alpha(G_1) \geq k_1\}$. Тогда любая задача $B \in \text{NP}$ полиномиально сводится к INDSET , которая полиномиально сводится к задаче распознавания, соответствующей **Maximum Acyclic Subgraph**, то есть решается за полиномиальное время на Маине Тьюринга с доступом к аракуду **Maximum Acyclic Subgraph**.

Итак, пусть нам дан неориентированный граф $G_1 = (V_1, E_1)$, хотим проверить, есть ли в нём независимое множество размера k_1 . Построим по нему следующий граф $G_2 = (V_2, E_2)$, где $V_2 = \{u_0 : u \in V_1\} \cup \{u_1 : u \in V_1\}$, а $E_2 = \{(u_0, u_1) : u \in V_1\} \cup \{(u_1, v_0) : (u, v) \in E_1\}$ (симметрично добавим и (v_1, u_0) , ибо $(v, u) \in E_1$). Пусть $n = |V_1|, m = |E_1|$. Тогда $|V_2| = 2n, |E_2| = n + 2m$. Положим $k_2 = 2m + k_1$. Утверждается, что в G_1 есть независимое множество размера $k_1 \iff$ в G_2 есть ациклический подграф размера k_2 .

Пусть в G_1 есть независимое множество $U = \{u^i\}_{i=1}^k$. Тогда рассмотрим подграф G_2 , образованный следующими рёбрами: $\{(u_1, v_0) : (u, v) \in E_1\} \cup \{(u_0^i, u_1^i) : u^i \in U\}$. Размер этого подграфа как раз $2m + k_1$. Предположим, что в этом подграфе есть цикл. Тогда в этом цикле нижние индексы вершин (0 или 1) чередуются, так как переход по каждому ребру в графе G_2 меняет нижний индекс вершины. Тогда в цикле обязательно встретится ребро, меняющее нижний индекс с 1 на 0, то есть ребро вида (u_1, v_0) , где $(u, v) \in E_1$. Но в вершину u_1 мы могли прийти только по ребру (u_0, u_1) , так как в вершины с нижним индексом 1 ведут только такие рёбра. Откуда ребро (u_0, u_1) содержится в подграфе, то есть $u \in U$. Аналогично из вершины v_0 можно выйти только по ребру (v_0, v_1) , а раз оно содержится в подграфе, то $v \in U$. То есть независимое множество U содержит и вершину u , и вершину v , но при этом $(u, v) \in E_1$. Противоречие, то есть построенный подграф ациклический.

Пусть в G_2 есть ациклический подграф содержащий k_2 рёбер. Тогда, если этот подграф не содержит какого-то ребра вида $\{(u_1, v_0) : (u, v) \in E_1\}$ (то есть ребра исходного графа из вершины с нижним индексом 1 в вершину с нижним индексом 0), то добавим в подграф это ребро, и удалим из подграфа ребро (u_0, u_1) , если оно в нём содержалось. Тогда размер подграфа не уменьшился. Покажем, что подграф остался ациклическим. Если в подграфе появился цикл, то он обязан проходить по ребру (u_1, v_0) , но, чтобы попасть в вершину u_1 , нам надо пройти по ребру (u_0, u_1) , так как это единственное ребро, которое ведёт в эту вершину. Противоречие, так как полученный подграф не содержит ребра (u_0, u_1) . Продолжая эту операцию, получим $G'_2 = (V_2, E'_2)$ - ациклический подграф графа G_2 размера

$k'_2 \geq k_2$, содержащий все рёбра вида $\{(u_1, v_0) : (u, v) \in E_1\}$. Можно считать, что он размера равно k_2 , удалив необходимое количество рёбер вида $\{(u_0, u_1) : u \in V_1\}$. Тогда определим множество вершин U графа G_1 следующим образом: $U = \{u \in V_1 : (u_0, u_1) \in E'_2\}$. Тогда размер U равен $k_2 - 2m = k_1$.

Предположим, что U не является независимым множеством, то есть $\exists u, v \in U : (u, v) \in E_1$. Тогда $(u, v) \in E_1 \implies (u_1, v_0) \in E'_2, (v_1, u_0) \in E'_2$ (так как E'_2 содержит все рёбра вида $\{(u_1, v_0) : (u, v) \in E_1\}$). Но раз $u \in U$, то $(u_0, u_1) \in E'_2$ (по определению множества U). Аналогично $v \in U \implies (v_0, v_1) \in E'_2$. Тогда в G'_2 есть цикл $u_0 \rightarrow u_1 \rightarrow v_0 \rightarrow v_1 \rightarrow u_0$. Противоречие с тем, что G'_2 - ациклический подграф G_2 , то есть U - независимое множество размера k_1 в графе G_1 .

Таким образом, мы показали, что в графе G_1 есть независимое множество размера k_1 ($\iff$ в G_1 есть независимое множество размера $\geq k_1$) $\iff$ в G_2 есть ациклический подграф размера $k_2 \iff$ максимальный ациклический подграф в G_2 имеет размер $\geq k_2$. Тогда так как любая задача из класса NP решается за полиномиальное время на Машине Тьюринга с аракулом $INDSET$, то и любая задача из NP решается за полиномиальное время на Машине Тьюринга с аракулом $Maximum Acyclic Subgraph \implies Maximum Acyclic Subgraph$ - NP -трудная задача.

3 Полиномиальный алгоритм аппроксимации с точностью $1/2$

Предъявим полиномиальный алгоритм, который позволяет находить ациклический подграф, содержащий не менее половины рёбер исходного графа. В частности, размер найденного ациклического подграфа будет не меньше половины от размера наибольшего ациклического подграфа.

Считаем, что зафиксирована произвольная нумерация вершин исходного графа. Тогда разделим рёбра исходного графа $G = (V, E)$ на 2 множества E_1 и E_2 . $E_1 = \{(u, v) \in E : u < v\}$, $E_2 = \{(u, v) \in E : u > v\}$. Так как в графе G петель нет по условию, то $E = E_1 \cup E_2$.

Покажем, что множество рёбер E_i образует ациклический подграф ($\forall i \in \{1, 2\}$).

Предположим, что подграф, образованный множеством рёбер E_1 , содержит цикл $u_1 \rightarrow u_2 \rightarrow u_3 \rightarrow \dots \rightarrow u_n \rightarrow u_1$. Но тогда $(u_1, u_2) \in E_1 \implies u_1 < u_2$, $(u_2, u_3) \in E_1 \implies u_2 < u_3 \dots (u_{n-1}, u_n) \in E_1 \implies u_{n-1} < u_n$. То есть $u_1 < u_2 < u_3 < \dots < u_{n-1} < u_n$, но $(u_n, u_1) \in E_1 \implies u_n < u_1$. Противоречие.

Аналогично если подграф, образованный множеством рёбер E_2 содержит цикл $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_n \rightarrow u_1$, то $u_1 > u_2 > \dots > u_n$, но $(u_n, u_1) \in E_2 \implies u_n > u_1$. Противоречие.

Откуда мы нашли 2 ациклических подграфа: образованный рёбрами из множества E_1 и образованный рёбрами из множества E_2 . Но E_1 и E_2 в объ-

единении дают E , откуда по принципу Дирихле $\exists i : |E_i| \geq \frac{1}{2}|E|$. Подграф, образованный рёбрами этого множества, и является ответом.

Приведём реализацию простейшего жадного алгоритма, описанного выше, на языке C++:

```
#include <vector>

struct Edge {
    int from;
    int to;
};

std::vector<Edge> AcyclicSubgraph(const std::vector<Edge>& graph) {
    std::vector<Edge> e1;
    std::vector<Edge> e2;
    for (auto edge : graph) {
        (edge.from < edge.to ? e1 : e2).push_back(edge);
    }
    if (e1.size() > e2.size()) {
        return e1;
    }
    return e2;
}
```

4 Улучшенный полиномиальный алгоритм аппроксимации с точностью $1/2$

Попробуем несколько улучшить наш жадный алгоритм. Для этого вспомним некоторые утверждения из курса Алгоритмов.

Опр. Топологической сортировкой ориентированного ациклического графа $G = (E, V)$ называется упорядочивание его вершин, такое что $\forall (u, v) \in E$: номер вершины u меньше, чем номер вершины v .

Утверждается, что в любом ориентированном ациклическом графе есть топологическая сортировка. Одним из простых способов это понятие является наивный алгоритм топологической сортировки.

В любом ориентированном ациклическом графе есть вершина, исходящая степень которой равна 0 (исходящая степень вершины - это число выходящих из неё рёбер). Действительно, если это не так, то из любой вершины выходит хотя бы одно ребро. Выберем произвольную вершину и пройдем по произвольному ведущему из неё ребру. Оказались в новой вершине, из неё также выходит ребро, пройдем по нему (если выходящих рёбер несколько - выберем любое). Вернутся в стартовую вершину мы не могли, так как это означало бы наличия цикла в исходном графе. Значит мы попали в новую вершину, пройдем по выходящему из неё ребру. И так далее. Каждый раз, оказываясь в новой вершине, выбираем произвольное ведущее из неё ребро

и проходим по нему. Вернутся по этому ребру в уже ранее посещённую вершину мы не можем, так как тогда мы нашли бы замкнутый маршрут, то есть цикл, в исходном графе. Тогда каждый раз мы оказываемся в новой вершине, но всего вершин конечное число, поэтому наш процесс не может продолжаться бесконечно долго, а, значит, рано или поздно мы придём в вершину, исходящая степень которой равна 0.

Тогда наивный алгоритм топологической сортировки заключается в следующем: находим вершину x с исходящей степенью 0, ставим её на последнюю позицию, а затем запускаемся рекурсивно от индуцированного подграфа, образованного оставшимися вершинами. Корректность этого алгоритма легко доказывается по индукции: для любого ребра исходного графа, не инцидентного вершине x условие выполнено по предположению индукции; любое ребро, инцидентное вершине x - это входящее в вершину x ребро (так как исходящая степень вершины x равна 0), для него условие выполнено, так как номер вершины x строго больше номера любой другой вершины графа. Базой индукции может служить случай, когда в исходном графе всего одна вершина.

Отметим, что симметричные рассуждения можно было провести и для входящих степеней вершин.

Наш текущий жадный алгоритм смотрит на некую перестановку вершин графа и оставляет либо только рёбра, ведущие слева направо, либо только рёбра, ведущие справа налево. Попробуем модифицировать его так, чтобы он рассматривал не произвольную перестановку, а некоторую особенную, специально нами построенную, и уже для этой перестановки оставим только рёбра, ведущие слева направо.

Для начала посчитаем для всех вершин нашего ориентированного графа их входящие и исходящие степени. Если в графе нашлась вершина исходящей степени 0, то присвоим ей номер $n - 1$ (где n - количество вершин исходного графа, нумерация вершин, которую мы строим, начинается с 0). Если в графе нашлась вершина входящей степени 0, то присвоим ей номер 0. Если же таких вершин не нашлось, то выберем ту вершину v , для которой величина $|\deg^-(v) - \deg^+(v)|$ максимальна, где $\deg^-(v)$ - это входящая степень вершины v , а $\deg^+(v)$ - исходящая степень вершины v . Если таких вершин (для которых $|\deg^-(v) - \deg^+(v)|$ максимальна) несколько, то можно выбрать любую из них, например, в моей реализации этого алгоритма берётся вершина с наибольшим номером в изначальной нумерации (изначальная нумерация - то, в каком порядке нам подавались на вход вершины графа). Если для этой вершины v $\deg^-(v) \geq \deg^+(v)$, то присвоим вершине v номер $n - 1$. Иначе, если $\deg^-(v) < \deg^+(v)$, то присвоим вершине v номер 0. Далее, удаляем вершину, которой мы уже присвоили номер и запускаемся рекурсивно от оставшегося индуцированного подграфа (в случае, если удалённой вершине присваивали номер 0, то, чтобы получить корректную нумерацию для исходного графа, нумерацию для индуцированного подграфа нужно будет сдвинуть на 1). Нетрудно видеть, приведённая часть алгоритма работает за полиномиальное время, так как мы совершили $O(m)$ итераций для подсчёта степеней вершин и уменьшили число вершин на 1

для рекурсивного запуска. Тогда итоговая асимптотика будет $O(nm)$.

Итак, мы получили некую нумерацию вершин исходного графа. Теперь выделим подграф, состоящий только из рёбер, ведущих из вершины с меньшим номером в вершину с большим номером (номера вершин берутся из построенной нумерации). Корректность нового жадного алгоритма (ацикличность построенного подграфа) следует из корректности простого жадного алгоритма.

Покажем, что построенный нашим модифицированным алгоритмом подграф содержит не менее половины рёбер исходного графа. Будем называть "выкинутым" ребро исходного графа, не включённое в подграф. Каждому ребру исходного графа сопоставим тот его конец, которому мы раньше присвоили номер. Рассмотрим множество всех рёбер, сопоставленных некоторой вершине. Достаточно показать, что среди этих рёбер мы выкинули не больше половины, и, тогда, в силу произвольности выбора вершины, всего выкинутых рёбер будет не больше половины.

Рассмотрим момент времени перед тем, как мы присвоили номер некоторой вершине v . В этот момент мы запускали наш рекурсивный алгоритм от некоторого индуцированного подграфа исходного графа (так как свойство индуцированности подграфа транзитивно, то мы всегда будем рекурсивно запускаться от некоторого индуцированного подграфа исходного графа). Этот индуцированный подграф содержит в точности те вершины, которым мы ещё не сопоставили номера, поэтому множество рёбер этого индуцированного подграфа, инцидентных вершине v - это в точности множество рёбер исходного графа, которым была сопоставлена вершина v . Если вершине v был присвоен наибольший из ещё не задействованных номеров (то есть наибольший номер из всех тех номеров, которые мы в дальнейшем будем присваивать другим вершинам этого индуцированного подграфа), то тогда $\deg^-(v) \geq \deg^+(v)$ (если вершина v была выбрана как вершина нулевой исходящей степени, то для неё $\deg^+(v) = 0$, поэтому неравенство автоматически выполнено). Но, заметим, что всем тем вершинам, из которых в вершину v входят рёбра индуцированного подграфа, будут присвоены номера, меньшие, чем номер v . А это значит, что при построении подграфа по нумерации все $\deg^-(v)$ рёбер, входящих в вершину v в индуцированном подграфе, будут включены в построенный ациклический подграф. Но всего вершина v сопоставлена $\deg^-(v) + \deg^+(v)$ рёбрам, а, так как $\deg^-(v) \geq \deg^+(v)$, то $\deg^-(v) \geq \frac{1}{2}(\deg^-(v) + \deg^+(v))$. То есть мы выкинули не более половины сопоставленных вершине v рёбер.

Аналогичные рассуждения проводятся и для случая, когда вершине v был присвоен наименьший из ещё не задействованных номеров. Тогда $\deg^+(v) \geq \deg^-(v)$, и при этом всем вершинам, в которые из вершины v выходят рёбра индуцированного подграфа, мы сопоставим номера, большие, чем номер вершины v . Но тогда все рёбра индуцированного подграфа, выходящие из вершины v , будут включены в итоговый ациклический подграф. А так как $\deg^+(v) \geq \deg^-(v)$, то $\deg^+(v) \geq \frac{1}{2}(\deg^-(v) + \deg^+(v))$, то есть выкидаем мы не более половины рёбер, которым была сопоставлена вершина v .

Нетрудно видеть, что модифицированный жадный алгоритм выдаёт наибольшее ациклический подграф в случае, если поданный ему на вход ориентированный граф уже был ациклическим. Это следует из того, что любой индуцированный подграф ациклического подграфа также ациклический, а значит в нём найдётся вершины входящей степени 0 или исходящей степени 0. Тогда просто по построению нашего алгоритма ближайшее присваивание номера будет именно такой вершине. И, повторяя рассуждения выше (и пользуясь тем, что $\deg^-$ или $\deg^+$ равны 0), получаем, что все сопоставленные этой вершине рёбра войдут в построенный ациклический подграф. Но это верно для любой итерации присваивания некоторой вершине номера (так как верно для любого индуцированного подграфа), а, значит все рёбра исходного графа войдут в построенный подграф.

5 Результаты запуска тестов

Для сравнения точности двух вышеперечисленных жадных алгоритма использовались следующие наборы тестов:

- 1) Случайные ориентированные графы с вероятностью p появления ребра. Здесь рассматривались $p \in \{0.15, 0.3, 0.4, 0.5, 0.6, 0.7, 0.85\}$
- 2) Всевозможные ориентированные графы на 5 вершинах (так как изолированные вершины никак не влияют на ответ, то рассматривание всевозможных ориентированных графов на ≤ 5 вершинах эквивалентно рассматриванию всевозможных ориентированных графов на 5 вершинах)
- 3) Турниры на n вершинах (рёбрам полного неориентированного графа присваивалась случайная ориентация).
- 4) На ориентированных ациклических графах. Такие графы строятся следующим образом: берётся случайная перестановка σ вершин графа. Если $\sigma^{-1}(i) \geq \sigma^{-1}(j)$, то ребро (i, j) отсутствует. Иначе $\sigma^{-1}(i) < \sigma^{-1}(j)$ и тогда ребро (i, j) проводится с вероятностью p (где $p \in \{0.15, 0.3, 0.4, 0.5, 0.6, 0.7, 0.85\}$). Ациклическость построенного графа следует из того, что σ^{-1} - его топологическая сортировка.
- 5) Графы, содержащие гамильтонов цикл. Бралась случайная перестановка вершин графа, добавлялись рёбра, для каждой вершины добавлялось ребро, ведущее в следующую вершину перестановки (первая вершина перестановки следует за последней). Все остальные рёбра в графе добавлялись или не добавлялись с вероятностью p , где $p \in \{0.0, 0.15, 0.3, 0.4, 0.5, 0.6, 0.7, 0.85\}$.
- 6) "Почти ациклические" ориентированные графы. Строятся такие графы так: берётся случайная перестановка σ вершин графа. Если $\sigma^{-1}(i) < \sigma^{-1}(j)$, то ребро (i, j) проводится с вероятностью p (где $p \in \{0.4, 0.5, 0.6, 0.7, 0.85\}$). Иначе $\sigma^{-1}(i) \geq \sigma^{-1}(j)$ и ребро (i, j) проводится с вероятностью p_{noise} , где $p_{noise} \in \{0.05, 0.1, 0.2, 0.3\}$. В случае, если бы p_{noise} был бы равен 0, мы бы получили в точности ориентированный ациклический граф, как в пункте (4).
- 7) Графы, которые получаются из ациклических добавлением в точности 1-го ребра (но при этом сами не являются ациклическими). Генерировались следующим образом: бралась случайная перестановка вершин, из каждой

вершины добавлялось ребро, ведущее в следующую вершину перестановки (первая вершина перестановки следует за последней), таким образом добавили гамильтонов цикл в граф. Затем добавляем остальные рёбра: ребро всегда ведёт от i -го элемента перестановки к j -му, где $i < j$. Каждое такое ребро (не содержащееся в изначальном цикле) проводится с вероятностью p . Таким образом, полученный подграф содержит цикл, и все рёбра, кроме одного, ведут из i -го элемента перестановки к j -му, где $i < j$, таким образом, можно удалить 1 ребро из исходного графа и получить ациклический граф.

8) Графы, которые получены объединением большого числа маленьких подграфов. Каждый из маленьких подграфов имеет размер n и является случайным ориентированным графом на n вершинах с вероятностью проведения ребра *probability inner*. Рёбра между подграфами ведут только из подграфа с меньшим номером в подграф с большим номером. Таким образом, минимальное число рёбер, которое надо удалить из графа, для того, чтобы он стал ациклическим, равно сумме по всем подграфам минимальных чисел рёбер, которые надо удалить из каждого подграфа для того, чтобы он стал ациклическим. Действительно, хотя бы это число рёбер удалить точно надо (так как добавление рёбер между подграфами не может уменьшить минимальное число рёбер, которое надо удалить для того, чтобы граф стал ациклическим). А при удалении нужного набора рёбер из каждого подграфа в полученном графе есть топологическая сортировка: топологически отсортируем каждый подграф и присвоим вершине с номером j в i -м подграфе номер $n \cdot i + j$. Нетрудно видеть, что это корректная топологическая сортировка полученного графа: рёбра внутри компонент ведут от вершины с меньшим номером в вершину с большим номером в силу корректности топологической сортировки для подграфа. А рёбра между компонентами ведут от вершины из компоненты с меньшим номером в вершину из компоненты с большим номером, в частности, ведут от вершины с меньшим номером в вершину с большим номером.

Для каждого из наборов тестов (и каждого из значений вероятности появления ребра, если она требовалась для генерации) высчитывалась метрика точности ассигасу, равная среднему отношению числа рёбер в подграфе, найденном жадным алгоритмом, к числу рёбер в наибольшем ациклическом подграфе. В наборах тестов (1) - (3), а также (5) - (6), истинный максимальный ациклический подграф искался за экспоненциальное время методами динамического программирования. В (4) сгенерированный граф был ациклическим, поэтому наибольший ациклический подграф совпадал с исходным графом. В (7) наборе тестов оптимальный ациклический подграф получается удалением одного ребра из исходного по условию генерации. В (8)-м наборе тестов ответ подсчитывается в процессе генерации описанным выше способом и возвращается из функции генерации вместе с полученным графом.

Размеры графов тоже выбирались случайно. В тестах, где требовалось явно искать максимальный ациклический подграф методами динамического программирования, рассматривались графы на 10 - 19 вершинах. На

группах тестов (4), (7), (8), в которых не требовалось вызывать явный алгоритм поиска максимального ациклического подграфа, рассматривались графы на 100 - 999 вершинах.

Ниже приведена таблица с результатами запуска тестов:

Таблица 1: Наивный жадный алгоритм

Категория тестов	Параметры (p)	Accuracy
Случайные ориентированные графы	p: 0.15	0.663815
	p: 0.30	0.735379
	p: 0.40	0.747341
	p: 0.50	0.783898
	p: 0.60	0.810957
	p: 0.70	0.854908
	p: 0.85	0.903905
Ориентированные графы на 5 вершинах		0.815808
Турниры		0.714625
Ациклические ориентированные графы	p: 0.15	0.514049
	p: 0.30	0.513008
	p: 0.40	0.513842
	p: 0.50	0.514063
	p: 0.60	0.512181
	p: 0.70	0.512565
	p: 0.85	0.512653
Графы с гамильтоновым циклом	p: 0.00	0.619103
	p: 0.15	0.685367
	p: 0.30	0.742923
	p: 0.40	0.769297
	p: 0.50	0.799452
	p: 0.60	0.838947
	p: 0.70	0.858099
	p: 0.85	0.907630
"Почти ациклические" ориентированные графы	p: 0.4, Noise: 0.05	0.643952
	p: 0.4, Noise: 0.10	0.656204
	p: 0.4, Noise: 0.20	0.704836
	p: 0.4, Noise: 0.30	0.733467
	p: 0.5, Noise: 0.05	0.622181
	p: 0.5, Noise: 0.10	0.660309
	p: 0.5, Noise: 0.20	0.699704
	p: 0.5, Noise: 0.30	0.731295
	p: 0.6, Noise: 0.05	0.605312
	p: 0.6, Noise: 0.10	0.661371
	p: 0.6, Noise: 0.20	0.703495
	p: 0.6, Noise: 0.30	0.743607

Продолжение таблицы 1

Категория тестов	Параметры (p)	Accuracy
	p: 0.7, Noise: 0.05	0.611942
	p: 0.7, Noise: 0.10	0.641509
	p: 0.7, Noise: 0.20	0.694081
	p: 0.7, Noise: 0.30	0.746308
	p: 0.85, Noise: 0.05	0.606805
	p: 0.85, Noise: 0.10	0.616783
	p: 0.85, Noise: 0.20	0.670931
	p: 0.85, Noise: 0.30	0.727469
	p: 0.15	0.512994
	p: 0.30	0.514592
	p: 0.40	0.511417
	p: 0.50	0.511291
Графы с одним удаляемым ребром	p: 0.60	0.512264
	p: 0.70	0.513318
	p: 0.85	0.513424
	p_in: 0.15, p_out: 0.15	0.509341
	p_in: 0.15, p_out: 0.30	0.507453
	p_in: 0.15, p_out: 0.50	0.507580
	p_in: 0.15, p_out: 0.70	0.508460
	p_in: 0.15, p_out: 0.85	0.508357
	p_in: 0.30, p_out: 0.15	0.509392
	p_in: 0.30, p_out: 0.30	0.508248
	p_in: 0.30, p_out: 0.50	0.507995
	p_in: 0.30, p_out: 0.70	0.507669
	p_in: 0.30, p_out: 0.85	0.508100
Большие графы	p_in: 0.50, p_out: 0.15	0.514133
	p_in: 0.50, p_out: 0.30	0.510489
	p_in: 0.50, p_out: 0.50	0.509086
	p_in: 0.50, p_out: 0.70	0.508877
	p_in: 0.50, p_out: 0.85	0.508928
	p_in: 0.70, p_out: 0.15	0.517964
	p_in: 0.70, p_out: 0.30	0.513036
	p_in: 0.70, p_out: 0.50	0.510962
	p_in: 0.70, p_out: 0.70	0.508933
	p_in: 0.70, p_out: 0.85	0.509596
	p_in: 0.85, p_out: 0.15	0.521911
	p_in: 0.85, p_out: 0.30	0.514154
	p_in: 0.85, p_out: 0.50	0.511381
	p_in: 0.85, p_out: 0.70	0.510471
	p_in: 0.85, p_out: 0.85	0.509777

Таблица 2: Улучшенный жадный алгоритм

Категория тестов	Параметры (p)	Accuracy
Случайные ориентированные графы	p: 0.15	0.983801
	p: 0.30	0.964529
	p: 0.40	0.963917
	p: 0.50	0.964581
	p: 0.60	0.972481
	p: 0.70	0.975509
	p: 0.85	0.987304
Ориентированные графы на 5 вершинах		0.994606
Турниры		0.942260
Ациклические ориентированные графы	p: 0.15	1.000000
	p: 0.30	1.000000
	p: 0.40	1.000000
	p: 0.50	1.000000
	p: 0.60	1.000000
	p: 0.70	1.000000
	p: 0.85	1.000000
Графы с гамильтоновым циклом	p: 0.00	1.000000
	p: 0.15	0.960502
	p: 0.30	0.960422
	p: 0.40	0.967265
	p: 0.50	0.969278
	p: 0.60	0.971445
	p: 0.70	0.976991
	p: 0.85	0.988542
"Почти ациклические" ориентированные графы	p: 0.4, Noise: 0.05	0.989576
	p: 0.4, Noise: 0.10	0.979310
	p: 0.4, Noise: 0.20	0.966618
	p: 0.4, Noise: 0.30	0.961548
	p: 0.5, Noise: 0.05	0.986677
	p: 0.5, Noise: 0.10	0.976938
	p: 0.5, Noise: 0.20	0.966594
	p: 0.5, Noise: 0.30	0.964979
	p: 0.6, Noise: 0.05	0.983904
	p: 0.6, Noise: 0.10	0.979835
	p: 0.6, Noise: 0.20	0.972896
	p: 0.6, Noise: 0.30	0.966239
	p: 0.7, Noise: 0.05	0.988892
	p: 0.7, Noise: 0.10	0.975398
	p: 0.7, Noise: 0.20	0.967814
	p: 0.7, Noise: 0.30	0.972705
	p: 0.85, Noise: 0.05	0.986253
	p: 0.85, Noise: 0.10	0.980997

Продолжение таблицы 2

Категория тестов	Параметры (p)	Accuracy
Графы с одним удаляемым ребром	p: 0.85, Noise: 0.20	0.975036
	p: 0.85, Noise: 0.30	0.980640
	p: 0.15	0.996465
	p: 0.30	0.997469
	p: 0.40	0.998712
	p: 0.50	0.999246
	p: 0.60	0.998960
	p: 0.70	0.999488
	p: 0.85	0.999794
	p_in: 0.15, p_out: 0.15	0.983742
	p_in: 0.15, p_out: 0.30	0.990814
	p_in: 0.15, p_out: 0.50	0.995086
	p_in: 0.15, p_out: 0.70	0.997789
	p_in: 0.15, p_out: 0.85	0.999089
Большие графы	p_in: 0.30, p_out: 0.15	0.970841
	p_in: 0.30, p_out: 0.30	0.981484
	p_in: 0.30, p_out: 0.50	0.989132
	p_in: 0.30, p_out: 0.70	0.994214
	p_in: 0.30, p_out: 0.85	0.997205
	p_in: 0.50, p_out: 0.15	0.963701
	p_in: 0.50, p_out: 0.30	0.976236
	p_in: 0.50, p_out: 0.50	0.984894
	p_in: 0.50, p_out: 0.70	0.991329
	p_in: 0.50, p_out: 0.85	0.995381
	p_in: 0.70, p_out: 0.15	0.961635
	p_in: 0.70, p_out: 0.30	0.973156
	p_in: 0.70, p_out: 0.50	0.982448
	p_in: 0.70, p_out: 0.70	0.989617
	p_in: 0.70, p_out: 0.85	0.994323
	p_in: 0.85, p_out: 0.15	0.961460
	p_in: 0.85, p_out: 0.30	0.972476
	p_in: 0.85, p_out: 0.50	0.981772
	p_in: 0.85, p_out: 0.70	0.988695
	p_in: 0.85, p_out: 0.85	0.993919

Точность наивного жадного алгоритма на случайных ориентированных графах сильно зависит от их плотности (то есть от вероятности ребра): чем выше вероятность появления ребра, тем больше средняя точность ассигасу. В целом точность ассигасу на случайных графах варьируется от 0.66 до 0.9. На маленьких графах точность в среднем довольно высокая (0.815). На турнирах точность 0.71, что ниже, чем на случайных графах с параметром 0.5, где точность 0.78. На ациклических ориентированных графах наивный жадный алгоритм работает очень плохо: точность едва превышает минимально возможную границу в 0.5. Причём точность почти не зависит от вероятности появления ребра. Наличие гамильтонова цикла в графе почти не влияет на точность наивного жадного алгоритма. На "почти ациклических" ориентированных графах наивный жадный алгоритм работает не очень хорошо: точность ассигасу порядка 0.6 - 0.74. Причём чем ниже шум (то есть чем ближе граф к ориентированному), тем ниже ассигасу.

Точность улучшенного жадного алгоритма очень высокая на всех наборах тестовых данных: на ациклических ориентированных графах она всегда в точности 1.0, как и было доказано выше. На остальных наборах тестов точность порядка 0.96-0.99. Единственный случай, на которых наш улучшенный алгоритм работает чуть хуже - это турниры, на них ассигасу в среднем 0.94.

На больших графах, сгенерированных по правилам, описанным выше, точность наивного жадного алгоритма близка к минимальной границе (чуть больше 0.5). То есть на довольно больших случайных графах, сгенерированных по правилам, описанным выше, при рассмотрении случайной перестановки вершин примерно половина всех рёбер смотрит в одну сторону, а другая половина - в другую сторону. А вот улучшенный жадный алгоритм показал превосходную точность в 0.96 - 0.99 на всех наборах тестов, даже для достаточно больших графов.

Итого, наш улучшенный алгоритм имеет значительно более высокую среднюю точность, чем наивный алгоритм. Точность улучшенного алгоритма в среднем близка к идеальной на всех наборах тестов, в то время как средняя точность наивного алгоритма зависит от категории теста и варьируется от 0.51 до 0.9.

Полный код обоих жадных алгоритмов, а также генерации тестов и оценки ассигасу можно найти по ссылке:

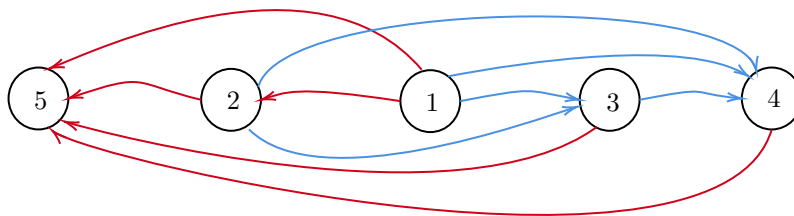
<https://gitlab.com/ekzlot/complexity-project.git>

6 Примеры

Пример, когда достигается теоретическая нижняя оценка точности наивного жадного алгоритма, привести не трудно. Так как этот алгоритм, как мы помним, включает в построенный ациклический подграф как минимум половину рёбер исходного графа. Для того, чтобы размер построенного ациклического подграфа был вдвое меньше оптимального ответа, нужно,

чтобы: во-первых, наш алгоритм выбирал ровно половину рёбер исходного графа (это происходит тогда, когда в заданной перестановке вершин слева-направо ведёт столько же рёбер, сколько и справа-налево), и, во-вторых, чтобы исходный подграф был ацикличен.

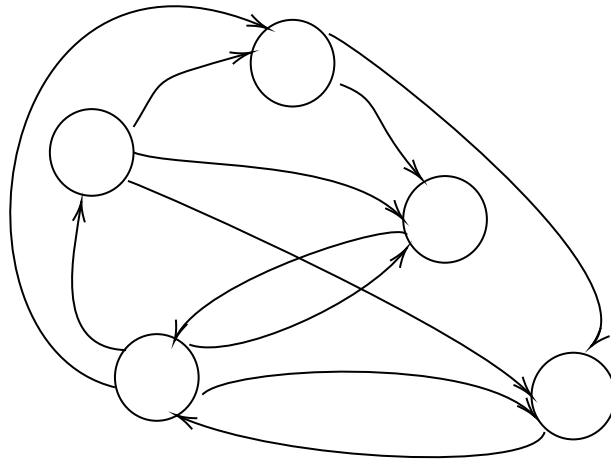
Ниже приведён пример такого графа на 5 вершинах. Наш наивный жадный алгоритм оставит ровно половину рёбер. Здесь считаем, что алгоритму поступают вершины в порядке "слева-направо", то есть вершина, помеченная 5 - это вершина номер 0, вершина помеченная 2 - это вершина номер 1 и так далее. Для наглядности, вершины помечены цифрами, соответствующими их номерам в топологической сортировке графа. То есть исходный граф ацикличен.



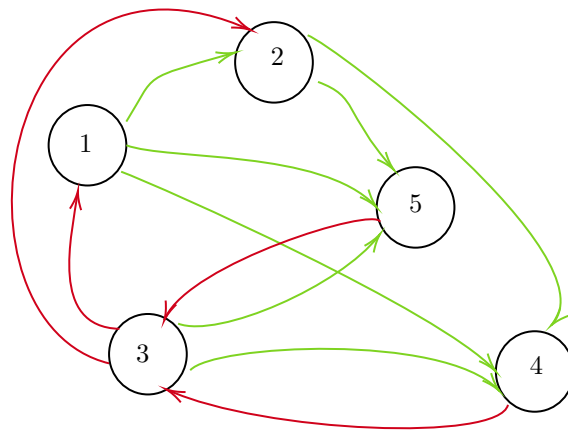
С улучшенным жадным алгоритмом ситуация совсем иная: нетрудно понять, что получить подграф размера в точности $1/2$ от максимального ациклического подграфа мы не можем, так как наш алгоритм по-прежнему оставляет хотя бы половину рёбер исходного графа. То есть ациклический подграф может содержать вдвое больше рёбер, только если он содержит все рёбра исходного графа. Но мы уже выяснили, что если исходный подграф был ацикличен, то наш улучшенный жадный алгоритм выдаст в точности оптимальный ответ.

И всё же, несмотря на то, что в среднем наш улучшенный жадный алгоритм работает очень хорошо и выдаёт высокую среднюю точность на всех наборах тестов (более 0.94), существуют примеры, когда его точность опускается значительно ниже этой границы.

Пример следующий:

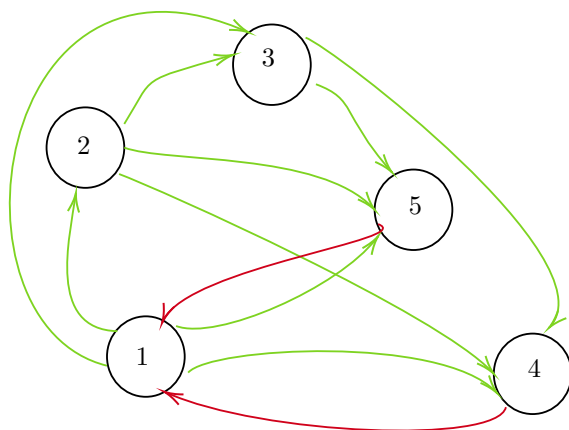


Давайте посмотрим на нумерацию вершин, найденную нашим улучшенным жадным алгоритмом (попытку приблизить топологическую сортировку максимального ациклического подграфа):



В этом примере наш жадный алгоритм присваивал вершинам номера в следующей последовательности: 1, 2, 3, 5, 4.

Из 11 рёбер исходного графа наш улучшенный жадный алгоритм смог оставить только 7. Утверждается, что на самом деле можно было оставить максимум 9 рёбер. То, что больше оставить нельзя, следует из того, что нужно "разорвать" непересекающиеся циклы $3 \rightarrow 5 \rightarrow 3$ и $3 \rightarrow 4 \rightarrow 3$. Вот пример, как оставить ровно 9 рёбер:



Точность нашего улучшенного жадного алгоритма на этом тесте получилась $\frac{7}{9} \approx 0.77778$.

Список литературы

- [1] B. Berger и P. W. Shor. “Approximation algorithms for the maximum acyclic subgraph problem”. В: *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '90. San Francisco, California, USA: Society for Industrial и Applied Mathematics, 1990, с. 236—243 (цит. на с. 2).
- [2] V. Guruswami, R. Manokaran и P. Raghavendra. “Beating the Random Ordering is Hard: Inapproximability of Maximum Acyclic Subgraph”. В: *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. 2008, с. 573—582 (цит. на с. 2).
- [3] R. M. Karp. “Reducibility Among Combinatorial Problems”. В: *50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art*. Под ред. M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi и L. A. Wolsey. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, с. 219—241. URL: https://doi.org/10.1007/978-3-540-68279-0_8 (цит. на с. 2).